Software Testing

ASSIGNMENT 2

CO2: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Duration:1week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.

S3	**Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.**
S4	**Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.**

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
 1  if weight <= 0 or weight > 50 then
 2    return INVALID
 3  rate = 40
 4  if zone == "B" then
 5    rate = 60
 6  else if zone == "C" then
 7    rate = 90
 8  charge = 0
 9  w = weight
10  while w > 0 do
11      charge = charge + rate
12    w = w - 1
13  if express then
14    if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

TASK 1 – Select Test Case Design Strategies 5 Marks

CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

- **The difference between black-box and white-box test design strategies**
- **The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states**
- **The information each strategy needs (specification, code, state model)**
- **The kinds of defects each strategy is likely and unlikely to reveal**
- **A justification for combining strategies rather than using only one**

You are expected to select and justify strategies independently for this system.

Evidence Expected

A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

TASK 2 – Evaluate Random Testing 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- **How random testing would be carried out for shipCharge**
- **The probability that random inputs hit important values such as 30.0, 30.1 and 50.0**
- **The problem of deciding the expected result (test oracle) for random inputs**

Your judgement must be supported by reasoning, not opinion.

Evidence Expected

A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

TASK 3 – Design and Evaluate Equivalence Classes 6 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- **Valid and invalid equivalence classes for weight, zone, express and parcels**
- **How you identified each class from the specification**
- **Test cases covering all valid classes with as few tests as possible**
- **Test cases covering each invalid class one at a time, and why**
- **Expected results calculated from the specification**
- **Which of your test cases would fail against the pseudocode, and what that reveals**

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

TASK 4 – Design and Evaluate Boundary Value Tests 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- **Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)**
- **Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces**
- **Test cases with expected results for the boundaries you select**
- **Which form of BVA you recommend for shipCharge, and why**

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

TASK 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- **A cause–effect graph or decision table for rules S3 and S4**
- **Test cases derived from the decision table**
- **Combinations that ECP/BVA did not cover, and whether they matter**

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

TASK 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- **A state-transition diagram and table for parcel tracking**
- **Test cases for all valid transitions and selected invalid transitions**
- **Tests for the "maximum two retries" rule and the cancellation rule**

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

TASK 7 – Construct the Control Flow Graph 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- **Nodes and edges, with line numbers mapped to nodes**
- **Decision nodes, the loop, and the compound condition at line 1**
- **Differences between the code structure and the rules S1–S4 (for example, invalid zones)**

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected

A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- **Cyclomatic complexity calculated by at least two methods**
- **A set of independent (basis) paths and test data for each path**
- **Infeasible paths, if any, and why**
- **How the loop affects path testing, and how you would test it (zero, one, many iterations)**

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

TASK 9 – Judge Test Adequacy and Coverage 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- **Statement, branch, condition and multiple-condition coverage for shipCharge**
- **The coverage achieved by your path tests from Task 8**
- **The additional tests needed to reach the next stronger criterion**
- **Whether 100 % coverage would have found the invalid-zone defect**

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

TASK 10 – Compare Black-Box and White-Box Effectiveness 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- **The defects found by black-box tests only, white-box tests only, and both**
- **Reasons why each approach missed what it missed**
- **Overlapping tests that can be removed without losing defect-finding power**
- **Whether this result would hold for other modules**

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

TASK 11 – Recommend and Defend the Final Test Suite 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- **The test cases you include and the technique each comes from**

- **Why this suite is better than the alternatives**
- **The coverage and the rule combinations it achieves**
- **The number of test cases and the effort needed to run them**
- **Known gaps and risks that remain**

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Test Suite 8 Marks

CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed in the specification and the code**
- **Which equivalence classes, boundaries and decision-table rules are affected**
- **How the control flow graph and cyclomatic complexity change**
- **Why the 30-minute limit matters**

Evidence Expected

A clear analysis of how the changes affect the original test design.

B. Evaluation of the Original Suite – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate.

Discuss:

- **Which existing tests are still valid, need new expected results, or are obsolete**
- **Which new paths and rule combinations are not covered**
- **Whether the coverage criterion can still be met**

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite.

If you modify or redesign it:

- **Describe the new and changed test cases**

- **Explain which techniques you used to derive them**
- **Show the effect on coverage and the number of test cases**

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

D. Evidence-Based Defence – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence.

Use:

- **Equivalence classes and boundaries**
- **Decision-table combinations**
- **Cyclomatic complexity and basis paths**
- **Coverage achieved**
- **Execution time limit**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Select Test Case Design Strategies	5
Task 2 – Evaluate Random Testing	2
Task 3 – Design and Evaluate Equivalence Classes	6
Task 4 – Design and Evaluate Boundary Value Tests	4
Task 5 – Apply Other Black-Box Techniques	3
Task 6 – Design and Evaluate State-Transition Tests	3
Task 7 – Construct the Control Flow Graph	4
Task 8 – Evaluate Path Testing and Cyclomatic Complexity	4

Task 9 – Judge Test Adequacy and Coverage	**3**
Task 10 – Compare Black-Box and White-Box Effectiveness	**4**
Task 11 – Recommend and Defend the Final Test Suite	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

ANSWERS

Assumptions used in the answers

- Weight is supplied to one decimal place. “Every started kilogram” means the number of chargeable kilograms is ceiling(weight).
- For the original specification, the discount is applied after the express surcharge. A parcel count below 10 receives no discount; 10 or more receives 10%.
- The specification does not define invalid values for express or parcels. Therefore, no unsupported invalid classes are invented for those inputs.
- For Task 12, the new 8% Zone C fuel surcharge and the weekend fee are assumed to be applied after the express calculation and before the existing 10% discount, because the assignment says the new decisions are inserted after line 16. The weekend fee is ₹50 multiplied by the parcels parameter. If the developer’s exact order differs, expected results must be adjusted.

TASK 1 – Select Test Case Design Strategies

Black-box testing derives tests from requirements without looking at source code. White-box testing derives tests from the internal code structure, such as decisions, branches, loops and execution paths. For SwiftShip, the most economical approach is to combine them rather than rely on only one.

Module / target	Recommended strategy	Information required	Strengths	Limitations
Shipping rules S1–S4	Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), and a decision table for interactions	Written specification and valid input ranges	Finds invalid ranges, threshold errors, incorrect rates, discount and express combinations	May miss internal unreachable code, loop defects or unmentioned requirements
shipCharge pseudocode	White-box statement, branch, condition and basis-path testing, supported by black-box expected results	Source code / pseudocode plus specification as the oracle	Finds unexecuted branches, incorrect control flow, loop problems and code/specification mismatches	Can execute every branch yet miss a requirement defect such as accepting an invalid zone
Parcel tracking	State-transition testing, including valid and invalid	State model and transition rules	Finds illegal state changes, cancellation errors, retry-limit	Does not by itself test charge calculations or

	transitions and transition sequences		errors and incorrect end states	numeric boundaries

For the charge calculator, ECP and BVA are best because the specification contains ranges and thresholds. A decision table is needed because express eligibility and the discount interact. For the implementation, the control-flow graph and basis paths show whether each decision and loop outcome is exercised. For tracking, a state model is the natural representation because the validity of an action depends on the current state. Combining these methods provides requirement-level coverage and implementation-level coverage.

Likely defects include incorrect weight limits, wrong zone rates, rounding a started kilogram down, applying express above 30 kg, applying the discount at 9 parcels, accepting an unknown zone, skipping a branch, or allowing cancellation after pickup. No single method is sufficient: white-box coverage cannot prove that requirements are complete, while black-box tests cannot prove that all internal branches have run.

TASK 2 – Evaluate Random Testing

Random testing can be performed by generating many independent input tuples: weight, zone, express flag and parcel count. Inputs should be generated from both valid and invalid ranges, and the expected result should be calculated by a separate trusted reference implementation of S1–S4. The random seed should be recorded so failures can be reproduced.

If weights are uniformly selected from the 500 one-decimal values 0.1 to 50.0, the probability of hitting one particular value such as 30.0, 30.1 or 50.0 in one trial is 1/500 = 0.2%. In 1,000 independent trials, the probability of hitting a particular value at least once is 1 − (499/500)^1000 ≈ 86.5%. This still leaves a meaningful chance of missing it. If weights are generated as continuous real numbers, the probability of hitting an exact decimal value is effectively zero.

The main difficulty is the test oracle: a random input is useful only if its correct result is known. A second implementation or independently calculated rule-based oracle is needed. Random testing is useful for broad exploration and unexpected combinations, but it cannot replace ECP, BVA or decision-table tests because important thresholds may be missed and random results may be difficult to reproduce. Use it as a supplement, not a replacement.

TASK 3 – Design and Evaluate Equivalence Classes

Equivalence classes group inputs expected to behave in the same way. The classes below are derived only from S1–S4. “Invalid” means explicitly rejected or disallowed by the specification.

Input	Equivalence class	Class type	Reason

Weight	w ≤ 0	Invalid	S1 rejects zero and negative weights
Weight	0 < w ≤ 50	Valid	S1 accepts this range
Weight	w > 50	Invalid	S1 rejects weights above 50
Zone	A	Valid	S2 assigns ₹40/kg
Zone	B	Valid	S2 assigns ₹60/kg
Zone	C	Valid	S2 assigns ₹90/kg
Zone	Any value other than A, B or C	Invalid	S2 says any other zone is invalid
Express	False	Valid	Standard delivery
Express	True and w ≤ 30	Valid	S3 permits express up to 30 kg
Express	True and w > 30	Invalid / not allowed	S3 returns NOT_ALLOWED
Parcels	p < 10	Valid; no discount	S4 threshold not reached
Parcels	p ≥ 10	Valid; 10% discount	S4 threshold reached

A compact valid-class set can use three tests because the three mutually exclusive zones each need one representative, while express false/true and parcel-count below/above 10 can be distributed across those tests. This is class coverage, not exhaustive combination coverage.

ID	**Weight**	**Zone**	**Express**	**Parcels**	**Expected result**
E1	1.0	A	False	9	₹40
E2	30.0	B	True	10	₹60 × 30 = ₹1,800; express = ₹2,700; discount = ₹2,430
E3	31.0	C	False	10	₹90 × 31 = ₹2,790; discount = ₹2,511

Invalid classes should be tested one at a time while keeping the other inputs valid. This isolates the reason for rejection.

ID	**Weight**	**Zone**	**Express**	**Parcels**	**Expected result**
I1	0.0	A	False	1	INVALID
I2	50.1	A	False	1	INVALID
I3	1.0	X	False	1	INVALID
I4	30.1	A	True	1	NOT_ALLOWED

The pseudocode returns INVALID for I1 and I2 and NOT_ALLOWED for I4. It does not reject I3: rate remains 40, so it calculates ₹40. This is a specification–code defect. The pseudocode

uses a default Zone A rate without validating that the zone is A, B or C. ECP is effective here because it explicitly creates an invalid-zone class. The parcel input has no stated invalid class in S1–S4; validating negative or fractional parcel counts would require an additional requirement.

TASK 4 – Design and Evaluate Boundary Value Tests

The highest-risk boundaries are weight just above zero, the 30 kg express limit, the 50 kg maximum, each whole-kilogram step in the started-kilogram rule, and the parcel discount threshold at 9/10.

Boundary / test	**Input example (other inputs fixed)**	**Expected result**
Lower weight limit	0.0 kg, Zone A, standard, 1 parcel	INVALID
Smallest valid weight	0.1 kg, Zone A, standard, 1 parcel	₹40 (one started kg)
Express limit	30.0 kg, Zone A, express, 1 parcel	₹1,800
Just above express limit	30.1 kg, Zone A, express, 1 parcel	NOT_ALLOWED
Maximum valid weight	50.0 kg, Zone A, standard, 1 parcel	₹2,000
Just above maximum	50.1 kg, Zone A, standard, 1 parcel	INVALID
Started-kilogram edge	1.0 kg then 1.1 kg, Zone A, standard	₹40 then ₹80
Discount edge	1.0 kg, Zone A, standard; parcels 9 then 10	₹40 then ₹36

Normal BVA for n numeric inputs commonly uses 4n + 1 tests (minimum, just above minimum, just below maximum, maximum, plus nominal values). Robust BVA adds just-outside values and commonly uses 6n + 1. Worst-case BVA combines all boundary values; for five values per variable it uses 5^n tests. Considering weight and parcel count as the two numeric variables gives 9 normal, 13 robust and 25 worst-case combinations. Zone is categorical and express is Boolean, so they should be handled with ECP/decision tables rather than blindly included in that numeric formula.

Robust BVA is recommended for weight because both invalid sides matter and the 30 kg express threshold is commercially important. Add focused tests for started kilograms and the 9/10 parcel boundary. Full worst-case combinations add many tests without proportionate value; a targeted robust set plus interaction tests is more efficient.

TASK 5 – Apply Other Black-Box Techniques

A decision table makes the interaction between express eligibility and the parcel discount explicit. “Weight ≤ 30” is relevant only when express is selected.

Rule	Express?	Weight > 30?	Parcels ≥ 10?	Expected action
R1	No	No	No	Standard charge; no discount
R2	No	Yes	Yes	Standard charge; apply 10% discount
R3	Yes	No	No	Add 50% express surcharge; no discount
R4	Yes	No	Yes	Add 50% express surcharge; then 10% discount
R5	Yes	Yes	No	Return NOT_ALLOWED
R6	Yes	Yes	Yes	Return NOT_ALLOWED; no charge/discount

Derived tests: (1) 2 kg, Zone A, standard, 9 parcels → ₹80; (2) 2 kg, Zone A, standard, 10 parcels → ₹72; (3) 2 kg, Zone A, express, 9 parcels → ₹120; (4) 2 kg, Zone A, express, 10 parcels → ₹108; (5) 30.1 kg, Zone A, express, 9 parcels → NOT_ALLOWED; (6) 30.1 kg, Zone A, express, 10 parcels → NOT_ALLOWED.

ECP and BVA establish representative classes and edges, but may not test all interactions between express, weight and discount. The decision table covers the meaningful combinations. The invalid express combination above 30 kg must take priority over the discount; the implementation returns before reaching the discount. This comparison shows why decision-table tests add value beyond isolated class and boundary tests.

TASK 6 – Design and Evaluate State-Transition Tests

The valid tracking model is: BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. From OUT_FOR_DELIVERY, a failed attempt may move to FAILED_ATTEMPT. From FAILED_ATTEMPT, the parcel may return to OUT_FOR_DELIVERY for at most two retries, or move to RETURNED. BOOKED may move to CANCELLED only before PICKED_UP.

Current state	Event / condition	Next state	Validity
BOOKED	Pickup confirmed	PICKED_UP	Valid
BOOKED	Cancel booking	CANCELLED	Valid
PICKED_UP	Dispatch	IN_TRANSIT	Valid
IN_TRANSIT	Arrives for final delivery	OUT_FOR_DELIVERY	Valid
OUT_FOR_DELIVERY	Successful delivery	DELIVERED	Valid
OUT_FOR_DELIVERY	Delivery failed	FAILED_ATTEMPT	Valid

FAILED_ATTEMPT	Retry, retry count within limit	OUT_FOR_DELIVERY	Valid
FAILED_ATTEMPT	Retry limit exceeded	RETURNED	Valid
FAILED_ATTEMPT	Return parcel	RETURNED	Valid

State-transition tests should traverse every valid edge at least once. Additional invalid tests should attempt BOOKED → DELIVERED, PICKED_UP → CANCELLED, DELIVERED → IN_TRANSIT, and a third retry after two permitted retries. These are high-risk because they can create false delivery records, cancel parcels already in the courier network, or cause repeated delivery attempts and extra operating cost. For retry testing, perform two failed attempts followed by the permitted retries, then verify that another retry is rejected or the parcel is returned. The wording "maximum two retries" should be clarified as whether the initial delivery attempt is excluded from the two retries.

TASK 7 – Construct the Control Flow Graph

The following node mapping is derived from the supplied pseudocode. Each decision has a true and false outgoing edge; the while loop has a body edge and an exit edge.

Node	**Pseudocode line(s)**	**Meaning**
N1	1	Check weight <= 0 OR weight > 50
N2	2	Return INVALID
N3	3–4	Set default rate 40; test zone B
N4	5	Set rate 60
N5	6	If zone B was false, test zone C
N6	7	Set rate 90
N7	8–9	Initialize charge and working weight
N8	10	while w > 0?
N9	11–12	Add rate; subtract 1 from w; return to N8
N10	13	Test express
N11	14	If express, test weight > 30
N12	15	Return NOT_ALLOWED
N13	16	Multiply charge by 1.5
N14	17	Test parcels >= 10
N15	18	Multiply charge by 0.9
N16	19	Return charge

Edge sequence: Start → N1. N1 true → N2 → End; N1 false → N3. N3 true → N4 → N7; N3 false → N5. N5 true → N6 → N7; N5 false → N7. N7 → N8. N8 true → N9 → N8; N8 false → N10. N10 false → N14; N10 true → N11. N11 true → N12 → End; N11 false → N13 → N14. N14 true → N15 → N16 → End; N14 false → N16 → End.

The compound condition at line 1 contains two atomic conditions joined by OR. The graph can show it as one decision for conventional control-flow complexity, while condition testing must exercise each atomic condition. The code does not match S2 completely: it initializes rate to Zone A and never checks that a zone is A, B or C. An unknown zone therefore receives the Zone A rate instead of INVALID. The code also needs a clear data contract for parcel count and the Boolean express input.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity

There are seven decision points in the conventional control-flow graph: the weight validation, zone B, zone C, while loop, express, weight > 30, and parcels >= 10. Therefore V(G) = decisions + 1 = 7 + 1 = 8.

Using V(G) = E − N + 2P, with one connected component (P = 1), the same graph gives 8 when its nodes and edges are counted. Thus eight independent basis paths are required for basis-path coverage. This is a minimum set of linearly independent paths, not a guarantee that every possible loop iteration count or input combination has been tested.

Basis path	**Representative test data**	**Path purpose / expected outcome**
P1	weight 0, A, standard, 1	Invalid-weight return
P2	weight 1, A, standard, 1	Zone B false, Zone C false; loop executes once; no express; no discount → ₹40
P3	weight 1, B, standard, 1	Zone B true; loop executes once → ₹60
P4	weight 1, C, standard, 1	Zone B false, Zone C true; loop executes once → ₹90
P5	weight 2, A, standard, 1	Loop executes multiple times → ₹80
P6	weight 1, A, express, 1	Express true, weight >30 false → ₹60
P7	weight 31, A, express, 1	Express true, weight >30 true → NOT_ALLOWED
P8	weight 1, A, standard, 10	Discount branch true → ₹36

A zero-iteration loop test is infeasible for valid input because S1 guarantees weight > 0; after initialization w is positive. The loop can be tested with one iteration (1.0 kg), multiple iterations (2.0 kg or 50.0 kg), and its eventual exit after the final subtraction. The loop is not safely represented by only one fixed path: test zero/one/many iterations where feasible, and use a maximum-weight case to check termination and upper-bound performance. The eight basis paths do not cover every possible weight, zone, express and parcel combination.

TASK 9 – Judge Test Adequacy and Coverage

Statement coverage checks whether every executable statement runs. Branch coverage checks both outcomes of every decision. Condition coverage checks true and false outcomes

of each atomic Boolean condition. Multiple-condition coverage checks combinations of atomic conditions; it is stronger and can grow rapidly.

The basis paths in Task 8 exercise the major decisions, both outcomes of the zone tests, the loop body and loop exit, express allowed/not-allowed outcomes, and discount/no-discount outcomes. The weight validation needs both invalid and valid tests. For the compound OR, include a low invalid weight (true, false), a high invalid weight (false, true), and a valid weight (false, false). The combination (true, true) is infeasible for a single numeric weight.

For a money calculation used on 40,000 parcels per day, recommend at least 100% branch coverage plus condition coverage for compound conditions, with ECP/BVA and decision-table tests. Multiple-condition coverage for the two-condition OR is small enough to analyse, but the true/true combination is infeasible. The basis-path set should be supplemented with invalid-zone and boundary tests. Importantly, even 100% statement or branch coverage would not automatically reveal the invalid-zone defect: if all tests use A, B or C, the default rate path can be fully covered while the missing validation remains undetected. A black-box test using zone X is necessary.

TASK 10 – Compare Black-Box and White-Box Effectiveness

For this module, the most important confirmed mismatch is that an unknown zone is not rejected. The black-box ECP test with zone X detects it because S2 explicitly says any other zone is INVALID. A white-box review also points to the missing validation: the code sets rate = 40 and only overrides it for B or C. However, executing ordinary basis paths with only valid zones will not expose the defect.

Defect / issue	Black-box contribution	White-box contribution
Unknown zone accepted as Zone A	Finds it through invalid-zone equivalence class	Code inspection identifies default-rate fall-through; execution needs an invalid-zone input
Incorrect threshold at 30 / 50 / 10	Finds it through BVA	Shows which comparisons and branches require execution
Loop off-by-one / termination	Started-kilogram expected results reveal wrong charges	Loop paths and iteration-count tests expose implementation errors
Unexecuted branch or dead code	May not reveal internal structure	Branch/path coverage identifies unexecuted code
Requirement not represented in code	Finds mismatch if requirement has a test	Code-only coverage cannot prove the requirement was implemented

Tests that overlap can be consolidated when one test simultaneously covers a valid zone, one loop iteration, express false and no discount. However, do not remove a test if it is the only one covering a distinct boundary, decision outcome or invalid class. Black-box tests are stronger for requirement conformance; white-box tests are stronger for internal logic and

path adequacy. This result may differ for the tracking module, where state-transition testing is more informative than a charge-calculation CFG.

TASK 11 – Recommend and Defend the Final Test Suite

Recommend a compact automated regression suite combining black-box and white-box tests. The following is a practical core suite; the test identifiers refer to the detailed cases already derived above.

Group	Included cases	Technique / purpose
Input classes	E1–E3 and I1–I4	ECP: all valid zone classes and important invalid classes
Numeric boundaries	0, 0.1, 30, 30.1, 50, 50.1; 1.0/1.1; parcels 9/10	Robust and targeted BVA
Interactions	R1–R6 decision-table cases	Express × weight eligibility × discount combinations
Code structure	P1–P8 basis paths plus loop 1/many iteration checks	White-box basis-path and loop testing
Tracking	All valid transitions; invalid cancellation, illegal completion and excess retry	State-transition testing

This suite is better than random-only testing because every important requirement class and threshold has a deliberate test and an expected result. It is better than a very large worst-case suite because it targets high-risk boundaries and meaningful interactions rather than multiplying every input dimension. It provides branch/path evidence as well as requirement-level evidence.

The exact execution time must be measured in the automated test environment. To remain within a 30-minute release window after Task 12, prioritize a fast core regression suite and run extended random/fuzz and exhaustive combinations outside the release gate.
Remaining risks include unspecified parcel-count validation, rounding/currency precision, concurrency, tracking event ordering, and any requirements not stated in the assignment.

TASK 12 – Re-evaluate and Adapt the Test Suite – Day-4 Design Challenge

A. Impact Analysis

Two new rules are introduced: Zone C receives an 8% fuel surcharge, and weekend delivery adds ₹50 per parcel. The code gains two decisions after the express calculation. The Zone C class now has an additional price rule, and weekend delivery creates a new Boolean input with weekend/non-weekend classes. New boundaries are not numeric thresholds in the supplied statement, but the amount is affected by parcel count because the fee is per parcel. The decision table must include Zone C versus non-Zone C and weekend versus weekday, including their interaction with express and discount.

The CFG gains two decision nodes and their associated branches. If each is a simple Boolean decision, cyclomatic complexity rises from 8 to 10. The 30-minute limit matters because an unrestricted cross-product of zone, express, weight, parcel threshold and weekend status can grow quickly. The release suite must focus on risk-based representative combinations and be automated.

B. Evaluation of the Original Suite

Original tests that exercise weight validity, the 30 kg express limit, the 50 kg limit, started-kilogram charging, the 9/10 discount threshold, and invalid zones remain relevant. Their expected results must be recalculated where the new surcharge or weekend fee applies. Zone A and B weekday tests can retain their original expected values if no new rule affects them. Zone C tests must change expected results to include the 8% surcharge. Every weekend test needs the ₹50-per-parcel fee. The original suite does not cover the two new decisions or their false/true branches and cannot retain full branch coverage of the modified code until those outcomes are exercised.

C. Adaptation / Redesign

Decision: MODIFY the original suite, rather than retain it unchanged or redesign it completely. Preserve the high-value ECP, BVA, state-transition and basis-path tests, update affected expected results, and add a small set of tests for the two new decisions.

New / modified test	**Input**	**Expected result under stated ordering assumption**
M1 – Zone C surcharge	1 kg, Zone C, weekday, standard, 1 parcel	Base ₹90; 8% surcharge = ₹7.20; total ₹97.20
M2 – No Zone C surcharge	1 kg, Zone A, weekday, standard, 1 parcel	₹40
M3 – Weekend fee	1 kg, Zone A, weekend, standard, 1 parcel	₹40 + ₹50 = ₹90
M4 – Zone C + weekend	1 kg, Zone C, weekend, standard, 1 parcel	₹90 + ₹7.20 + ₹50 = ₹147.20
M5 – New rules with discount	1 kg, Zone C, weekend, standard, 10 parcels	Per-parcel base+surcharge+weekend = ₹147.20; ×10 = ₹1,472; 10% discount = ₹1,324.80
M6 – Express plus new rules	2 kg, Zone C, express, weekend, 10 parcels	Base ₹180; express ₹270; Zone C surcharge ₹21.60; weekend ₹500; subtotal ₹791.60; discount = ₹712.44

M1–M4 cover both outcomes of each new decision and their interaction. M5 and M6 check that the additions are included before the existing discount and that the new rules coexist with the existing express and discount rules. The modified CFG has complexity 10, so at least two additional independent paths are required in a basis set. In practice, add tests that independently take each new decision true and false, while retaining the old basis paths. The suite should be automated and kept to a compact release set; measure actual execution time and remove only redundant tests, not unique boundary or branch coverage.

D. Evidence-Based Defence

The modified suite is justified by the original equivalence classes and boundaries, the S3/S4 decision table, and the two added surcharge/weekend decisions. It retains tests for invalid weights, invalid zones, express at 30.0/30.1 kg, maximum weight 50.0/50.1 kg, started kilograms and parcels 9/10. It adds Zone C surcharge and weekend true/false paths, plus combined tests to verify ordering with the discount. Cyclomatic complexity increases from 8 to 10, so the basis-path evidence must also increase by two independent paths. Branch and condition coverage should be re-measured against the modified code; the original coverage claim cannot simply be carried forward.

The final release suite should be the smallest suite that preserves unique requirement classes, critical boundaries, new rule combinations and modified-code branch outcomes. Run it automatically on every release and enforce the 30-minute limit using measured execution time. Run broader random testing and extended combinations outside the release gate. The suite still cannot prove unspecified behaviour such as currency rounding policy, negative parcel-count handling or concurrency; these need explicit requirements and separate tests.